



university of
groningen

Models and Semantics of Computation (MaSC)

Lecture 1: Introduction

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI
University of Groningen, Groningen, the Netherlands



Part I

Logistics

Overview



MaSC examines a central problem in Computer Science: How to **assign meaning** to programs.



MaSC examines a central problem in Computer Science: How to **assign meaning to programs**.

Understanding program semantics is crucial for computing professionals, who are confronted with programming languages from different paradigms on a daily basis.

Here we study **sequential** and **concurrent** models of computation by developing their underlying semantic models and establishing important properties.

Learning Outcomes (Ocasys)



At the end of the course, you will be able to:

1. Identify the distinguishing features of rigorous models of computation for imperative, functional, and concurrent programming.
2. Develop abstract specifications of programs in different models of computation, and identify the key differences between them.
3. Define and establish basic properties of models of computation using precise mathematical proofs.
4. Demonstrate the use of behavioral equivalences for the analysis of specifications in a process calculus.

Learning Outcomes (Ocasys)



At the end of the course, you will be able to:

1. Identify the distinguishing features of rigorous models of computation for imperative, functional, and concurrent programming.
2. Develop abstract specifications of programs in different models of computation, and identify the key differences between them.
3. Define and establish basic properties of models of computation using precise mathematical proofs.
4. Demonstrate the use of behavioral equivalences for the analysis of specifications in a process calculus.

Ideally:

After MaSC, you will be prepared (and motivated!) to embark into a research thesis on the semantics of programming languages.



We assume that you know and are comfortable with topics such as:

- **Discrete Mathematics:**

Sets, relations (and their properties), the induction principle (and proofs by induction), ordering relations

- **Logic:**

Formal reasoning in propositional logic and first-order logic

- **Automata Theory:**

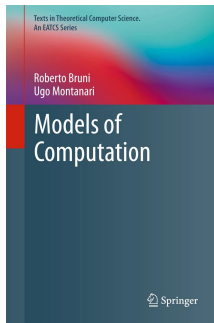
Strings, grammars, regular expressions, finite-state machines



1. A taste of semantic methods (today!)
2. Inference and unification
3. Well-founded induction
4. IMP, a core imperative language
5. Rule induction
6. IMP: Abstract and denotational semantics
7. HOFL, a core functional language
8. CCS, the Calculus of Communicating Systems
9. The π -calculus: A calculus of concurrency and distribution
10. Type systems for concurrency

Material

Textbook:



- *Models of Computation* by Roberto Bruni and Ugo Montanari.
- Course slides based on those by Roberto Bruni (used with permission)

Many other excellent resources (textbooks, tutorials) around!



On our side

A combination of **lectures** and **tutorials**.

A typical week, four meeting points:

- Monday: Lecture and tutorial
- Thursday: Lecture and tutorial

Tutorials are **flexible**: We can use them to discuss homework exercises, and/or to have individual meetings on the project. This is entirely up to you.

There are variations. Check “Schedule 2026” in Brightspace for updates.



On our side

A combination of **lectures** and **tutorials**.

A typical week, four meeting points:

- Monday: Lecture and tutorial
- Thursday: Lecture and tutorial

Tutorials are **flexible**: We can use them to discuss homework exercises, and/or to have individual meetings on the project. This is entirely up to you.

There are variations. Check “Schedule 2026” in Brightspace for updates.

On your side

Self-study! Read in advance before each lecture.

Grading Components



Assignments (40 %)

- ▶ Goal: Apply the concepts seen in class
- ▶ Three sets of individual exercises
- ▶ Released every two weeks

Project (60 %)

- ▶ Goal: Read, analyze, and present a well-known research paper
- ▶ Developed throughout the quarter, with frequent checkpoints
- ▶ Three components: Proposal, report, presentation
- ▶ Done individually, exceptions possible

Check Ocasys for details.

The use of AI, in any of its wonderful manifestations, is **strictly forbidden**.

Deadlines



There is roughly **one deadline per week**, either for homework or for the project:

Week 2 Deadline project proposal.

Week 3 Deadline HW1.

Week 4 Deadline project report (first draft).

Week 5 Deadline HW2.

Week 6 Deadline project report (second draft).

Week 7 Project checkpoint (optional, but recommended).

Week 8 Deadline HW3.

Week 9 Deadline project: report (final version) and slides.

Week 10 Final presentations. Attendance is mandatory.

Note:

- The project **proposal** has its own grade.
- The project **report** is graded on the basis of the drafts and the process.

MaSC and BaSC



- ▶ MaSC is the graduate counterpart of BaSC, a third-year elective course for BSc students (1B).

MaSC and BaSC



- ▶ MaSC is the graduate counterpart of BaSC, a third-year elective course for BSc students (1B).
- ▶ Although having passed BaSC is useful experience, it is not a strict pre-requisite for succeeding in MaSC.



- ▶ MaSC is the graduate counterpart of BaSC, a third-year elective course for BSc students (1B).
- ▶ Although having passed BaSC is useful experience, it is not a strict pre-requisite for succeeding in MaSC.
- ▶ There is an overlap in topics, but MaSC goes beyond BaSC by covering additional topics: models of concurrency (CCS and the π -calculus)
- ▶ Also, MaSC focuses on research-oriented learning goals by means of the individual project.

Scientific Integrity and Plagiarism



Study Guide Computer Science, section on “Fraud prevention & scientific integrity” ([URL](#)):

Plagiarism is not accepted at this university nor elsewhere in the scientific community.

In all cases in which plagiarism is found or suspected, the examiner will inform the Board of Examiners.



Part II

A Taste of Semantic Methods



When we **define a programming language**, we fix its:

Syntax	Well-formed programs, exclude nonsense
Types	Reduce allowed programs, exclude common mistakes
Pragmatics	How to use constructs and features
Semantics	The meaning of (well-typed) programs

Formal Syntax



The syntax of a formal language rigorously defines

1. The **alphabet**: which symbols can be used
2. The **grammatical structure** of programs: which sequences of symbols are valid, which sequences should be discarded

Formal Syntax



The syntax of a formal language rigorously defines

1. The **alphabet**: which symbols can be used
2. The **grammatical structure** of programs: which sequences of symbols are valid, which sequences should be discarded

Standard ways for defining syntax: regular expressions, context-free grammars, BNF notation, syntax diagrams, ...

Formal Syntax



The syntax of a formal language rigorously defines

1. The **alphabet**: which symbols can be used
2. The **grammatical structure** of programs: which sequences of symbols are valid, which sequences should be discarded

Standard ways for defining syntax: regular expressions, context-free grammars, BNF notation, syntax diagrams, ...

Example

A BNF grammar:

$$\begin{aligned} \langle \text{numeral} \rangle &::= \langle \text{digit} \rangle \mid \langle \text{numeral} \rangle \langle \text{digit} \rangle \\ \langle \text{digit} \rangle &::= "0" \mid "1" \mid "2" \mid "3" \mid "4" \mid "5" \mid "6" \mid "7" \mid "8" \mid "9" \end{aligned}$$



Type systems can be used to

1. Limit the occurrence of errors
2. Allow compiler optimizations
3. Reduce the presence of bugs
4. Discourage programming malpractices

Type systems are often presented as **logic rules**

Type Systems



Type systems can be used to

1. Limit the occurrence of errors
2. Allow compiler optimizations
3. Reduce the presence of bugs
4. Discourage programming malpractices

Different type systems
can be defined for
the same language!

Type systems are often presented as **logic rules**

Benefits of Formalization



- ▶ **Standardisation of the language**
programmers write syntactically correct programs
implementors write correct parsers
- ▶ **Formal analysis of language properties**
ambiguity, expressiveness, recognizability, comparability
- ▶ **Automatic implementation of compiler's front-end**
yacc, Bison, Xtext, etc



Programmers should understand the code they type!

Every language manual also contains

1. natural language descriptions of the various constructs
2. sample code fragments and usage patterns
3. examples of malpractices



Programmers should understand the code they type!

Every language manual also contains

1. natural language descriptions of the various constructs
2. sample code fragments and usage patterns
3. examples of malpractices

We call them **pragmatics**:

1. how to exploit the various features
2. how compilers should be designed
3. which auxiliary tools are available

Is it enough?



Natural language descriptions should be

1. as much precise as possible
2. understandable
3. unambiguous but not pedantic

Is it enough?



Natural language descriptions should be

1. as much precise as possible
2. understandable
3. unambiguous but not pedantic

Still, there are issues:

1. it is difficult (nearly impossible) to cover all cases
2. many points will remain open to different interpretations
3. inconsistencies can arise
4. good practices target problems but do not eliminate them (they hide them!)

Some Persistent Issues

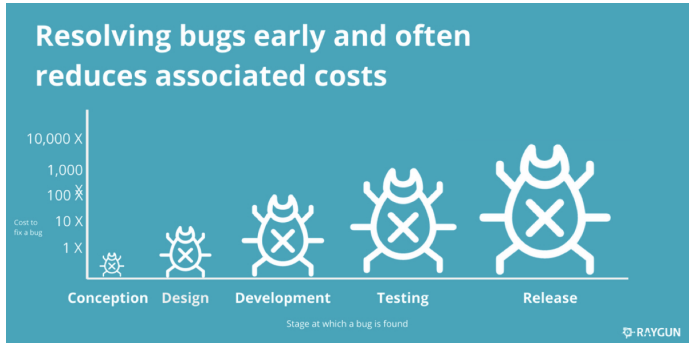


- ▶ How to prove conformance to some specification?
- ▶ How to prove absence of problems?
- ▶ How to produce reliable code?
- ▶ How to prove vendors' compliance?
- ▶ How to prove correctness of an implementation?
- ▶ How to define the correct outcomes of test cases?
- ▶ How to early detect ambiguities, anomalies, inconsistencies?
- ▶ How to expose weaknesses?
- ▶ ...

Some Persistent Issues



Practices such as code reviews and test-driven development do not really address the fundamental issues:





- ▶ The word **semantics** was introduced in 1900 as
the study of how words change their meanings (Michel Bréal)
- ▶ Ironically, its meaning has now changed to
the study of the attachment between the sentences of a language (written, spoken or formal) and their meanings
- ▶ In Computer Science, it is concerned with
the study of the meaning of (well-typed) programs

Formal semantics assigns rigorous, non-ambiguous meaning: it tells programmers the meaning of the code they type (at some level of abstraction)



Someone may always claim

- ▶ “(my) implementation is the semantics of the language”

It is correct by definition, but



Someone may always claim

- ▶ “(my) implementation is the semantics of the language”

It is correct by definition, but

- ▶ machine-independent?
- ▶ portability?
- ▶ how long it will last?
- ▶ useful abstraction for other programmers?
- ▶ how to reason on it?
- ▶ what about competitors?

Benefits of Formalization (Again)



- ▶ **Standardization of the reference model of the language**
official, machine-independent
a mental model for programmers
a benchmark for implementors
- ▶ **Formal analysis of language properties**
subtleties, expressiveness, type safety,
program compliance, subject reduction
- ▶ **Automatic implementation of compiler's front-end**
prototypical interpreter for experimentation

Math and logic? I want to code!



FT Weekend

Global IT outage throws travel, payments and health into chaos



(July 19, 2024)

The New York Times

What Happened to Digital Resilience?

With each cascade of digital disaster, new vulnerabilities emerge. The latest chaos wasn't caused by an adversary, but it provided a road map of American vulnerabilities at a critical moment.

(July 19, 2024)



Roughly, semantics definition methods fall into three groups:

- ▶ **Operational**
The interest is in **how** the effect of the computation is achieved
- ▶ **Denotational**
Here only the **effect** is of interest, not how it is obtained
- ▶ **Axiomatic**
Focus is on valid **assertions** about the computation (Hoare logic, anyone?)



- ▶ **Idea:**
Define some kind of **abstract machine** and describe the meaning of a program in terms of the steps that this machine executes to perform the task
- ▶ **Rationale:** Explain computations

$$s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_n \rightarrow r$$

Important aspects:

- ▶ We study 'abstract machines' as opposed to concrete computing devices
- ▶ The emphasis is on states and state transformations

Operational Semantics: History



- ▶ ['70s] Small-step:

Semantics of LISP by John McCarthy (1960) and of Algol 68 (1975)

- ▶ ['80s] SOS approach:

Gordon Plotkin introduced the structural operational semantics in 1981.
'Structural': syntax-oriented and inductively defined.

$$\frac{\langle e_0, \sigma \rangle \rightarrow \langle e'_0, \sigma \rangle}{\langle e_0 + e_1, \sigma \rangle \rightarrow \langle e'_0 + e_1, \sigma \rangle}$$

- ▶ ['90s] Big-step:

Gilles Kahn introduced the natural semantics in 1987, where the result is computed in a single step:

$$s \longrightarrow r$$

SOS Semantics: Overview



The transition relation between states is typically defined inductively, by axioms and inference rules according to the syntax of the program.

SOS Semantics: Overview



The transition relation between states is typically defined inductively, by axioms and inference rules according to the syntax of the program.

Advantages:

- ▶ immediate implementation (Horn clauses in logic programming);
- ▶ prototype Prolog interpreter (almost) for free;
- ▶ strong connections to the syntax of the language;
- ▶ rules for different constructs are neatly separated;
- ▶ useful to detect underspecified behaviors;
- ▶ involved mathematics is usually not much complicated;
- ▶ SOS descriptions are easy to read, even for non-specialists;
- ▶ could appear in any manual (but usually it doesn't)



► Idea:

A program's meaning is some mathematical object (e.g. a function from input to output) and the steps taken to calculate the result are unimportant:

$$\llbracket \cdot \rrbracket : \textit{Programs} \rightarrow \textit{Domain}$$

► Rationale:

Functions are independent of their means of computation; hence, they are simpler than the step-by-step sequence in an operational semantics



- ▶ ['70s]: Christopher Strachey and Dana Scott

Compositionally principle: the semantics takes the form of a function that assigns an element of some mathematical domain to each individual construct.

Key principle:

the meaning of a composite construct does not depend on the particular form of the constituent constructs, but only on their meanings

Denotational Semantics: Overview



Advantages:

- ▶ mathematically elegant;
- ▶ useful to detect underspecified behaviors;
- ▶ can be used to derive prototype implementations;
- ▶ has served as inspiration for many programming languages;
- ▶ difficult to apply to concurrent, interactive systems



► Idea:

Describe the constructs in a programming language by providing logical axioms that are satisfied by these constructs

► Rationale:

Prove the correctness of a program with respect to a given specification

$$\vdash \{P\} c \{Q\}$$



- ['60s]: Robert W. Floyd (1967) and Tony Hoare (1969)

Hoare logic: A statement is accompanied by a precondition (the state before the execution) and a postcondition (after the execution).

Key principle:

the meaning of a program is a logical proposition that states some property of the output whenever some properties of the input are met

Axiomatic Semantics: Overview



Advantages:

- ▶ emphasis on proof correctness from the very start;
- ▶ strikingly elegant proof systems;
- ▶ can be used to prove absence of bugs;
- ▶ difficult to apply to concurrent, interactive systems

Which One to Choose?



- ▶ Different semantics are often seen in opposition one each other, but this should not be the case! We would gain much more from their combination!
- ▶ In this course, we focus on **operational** and **denotational** semantics
- ▶ We will present the fundamental ideas behind these approaches and stress their relationship, by proving some relevant correspondence theorems.

A Simple Language



Informal syntax of **numerical expressions**:

- ▶ any numeral N is an expression;
- ▶ if E_1 and E_2 are expressions, then $E_1 \oplus E_2$ is an expression;
- ▶ if E_1 and E_2 are expressions, then $E_1 \otimes E_2$ is an expression.

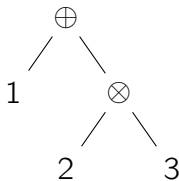
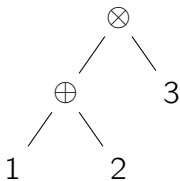
Numerals vs numbers:

- ▶ Numerals is a syntax (text) for writing numbers ('5', '1 0 1', 'five', 'cinq')
- ▶ Numbers, denoted $n, n', \dots$, denote mathematical objects (i.e. concepts).



$$E ::= N \mid E_1 \oplus E_2 \mid E_1 \otimes E_2$$

- ▶ The string ' $\oplus 3 \otimes 4$ ' is **not a well-formed** numerical expression.
- ▶ The string ' $1 \oplus 2 \otimes 3$ ' is **well-formed**. Two **abstract syntax trees**:



- ▶ To solve ambiguities, we use parenthesis (e.g., ' $1 \oplus (2 \otimes 3)$ ') or fix precedence between operators.

Assigning Meaning to Expressions



$$E ::= N \mid E_1 \oplus E_2 \mid E_1 \otimes E_2$$

Going beyond syntax:

- ▶ is N necessarily a number?
- ▶ is $\oplus$ necessarily the arithmetic sum?
- ▶ is $\otimes$ necessarily the arithmetic product?

Assigning Meaning to Expressions



$$E ::= N \mid E_1 \oplus E_2 \mid E_1 \otimes E_2$$

Going beyond syntax:

- ▶ is N necessarily a number?
- ▶ is $\oplus$ necessarily the arithmetic sum?
- ▶ is $\otimes$ necessarily the arithmetic product?

Some possibilities—maybe we mean

- ▶ matrices with addition and multiplication,
- ▶ or sets with union and intersection
- ▶ or strings with concatenation and least common prefix
- ▶ or trees with branching and merging

An Informal Semantics



- ▶ a numeral N evaluates to its corresponding number n ;

An Informal Semantics



- ▶ a numeral N evaluates to its corresponding number n ;
- ▶ to evaluate $E_1 \oplus E_2$ we evaluate E_1 and E_2 and sum their values;

An Informal Semantics



- ▶ a numeral N **evaluates** to its corresponding number n ;
- ▶ to evaluate $E_1 \oplus E_2$ we evaluate E_1 and E_2 and sum their values;
- ▶ to evaluate $E_1 \otimes E_2$ we evaluate E_1 and E_2 and multiply their values.



- ▶ a numeral N **evaluates** to its corresponding number n ;
- ▶ to evaluate $E_1 \oplus E_2$ we evaluate E_1 and E_2 and sum their values;
- ▶ to evaluate $E_1 \otimes E_2$ we evaluate E_1 and E_2 and multiply their values.

Note:

- ▶ These three rules are enough to determine the value of any well-formed expression, no matter how large
- ▶ We are not specifying the order in which arguments are evaluated: is it important?



Example

- ▶ 2 evaluates to 2
- ▶ $(1 \oplus 2) \otimes 3$ evaluates to 9
- ▶ $(1 \oplus 2) \otimes (3 \oplus 4)$ evaluates to 21

Small-step Semantics



Let's define a syntax of **runtime** numerical expressions:

$$E ::= n \mid N \mid E_1 \oplus E_2 \mid E_1 \otimes E_2$$

Intuition: The state of the abstract machine can mix intermediate results with (not yet evaluated) expressions

Small-step Semantics



Let's define a syntax of **runtime** numerical expressions:

$$E ::= n \mid N \mid E_1 \oplus E_2 \mid E_1 \otimes E_2$$

Intuition: The state of the abstract machine can mix intermediate results with (not yet evaluated) expressions

We want a **small-step** semantics, such that:

- A step:

$$E_0 \rightarrow E_1$$

- An evaluation:

$$E_0 \rightarrow E_1 \rightarrow E_2 \rightarrow \cdots \rightarrow E_k \rightarrow n \quad (\text{also written } E_0 \rightarrow^* n)$$

- We also expect $n \not\rightarrow$

Inference Rules: Format



(Rule Name)

$$\frac{\text{premise}_1 \quad \dots \quad \text{premise}_n}{\text{conclusion}} \quad (\text{side condition})$$

Informally: If the premises and the side condition are met then the conclusion holds.

Inference Rules: Format


$$\begin{array}{c} \text{(Rule Name)} \\ \text{premise}_1 \quad \dots \quad \text{premise}_n \quad \text{(side condition)} \\ \hline \text{conclusion} \end{array}$$

Informally: If the premises and the side condition are met then the conclusion holds.

Note:

- ▶ The conclusion is a single **judgement**
- ▶ The premises consist of one, none or more judgements
- ▶ The side condition is a **logical predicate**
- ▶ The rule name is just a convenient label

SOS Rules - Small-step Semantics



$$\begin{array}{c} \text{(num)} \\ \hline N \rightarrow n \end{array}$$

$$\begin{array}{c} \text{(sum)} \\ \hline n_0 \oplus n_1 \rightarrow n \end{array} \quad n = n_0 + n_1$$

$$\begin{array}{c} \text{(sumL)} \\ E_0 \rightarrow E'_0 \\ \hline E_0 \oplus E_1 \rightarrow E'_0 \oplus E_1 \end{array}$$

$$\begin{array}{c} \text{(sumR)} \\ E_1 \rightarrow E'_1 \\ \hline E_0 \oplus E_1 \rightarrow E_0 \oplus E'_1 \end{array}$$

SOS Rules - Small-step Semantics



$$\frac{(\text{num})}{N \rightarrow n}$$

$$\frac{(\text{sum})}{n_0 \oplus n_1 \rightarrow n} \quad n = n_0 + n_1$$

$$\frac{(\text{sumL}) \quad E_0 \rightarrow E'_0}{E_0 \oplus E_1 \rightarrow E'_0 \oplus E_1}$$

$$\frac{(\text{sumR}) \quad E_1 \rightarrow E'_1}{E_0 \oplus E_1 \rightarrow E_0 \oplus E'_1}$$

What about the rest?

(prod)

(prodL)

(prodR)

Some Derivations



Example

$$\begin{array}{c} \text{(num)} \frac{}{1 \rightarrow 1} \\ \text{(sumL)} \frac{}{(1 \oplus 2) \rightarrow (1 \oplus 2)} \\ \text{(prodL)} \frac{}{(1 \oplus 2) \otimes (3 \oplus 4) \rightarrow (1 \oplus 2) \otimes (3 \oplus 4)} \end{array}$$

Some Derivations



Example

$$\begin{array}{c} \text{(num)} \frac{}{2 \rightarrow 2} \\ \text{(sumR)} \frac{}{(1 \oplus 2) \rightarrow (1 \oplus 2)} \\ \text{(prodL)} \frac{}{(1 \oplus 2) \otimes (3 \oplus 4) \rightarrow (1 \oplus 2) \otimes (3 \oplus 4)} \end{array}$$

Example

$$\begin{array}{c} \text{(sum)} \frac{}{1 \oplus 2 \rightarrow 3} \quad 3 = 1 \oplus 2 \\ \text{(prodL)} \frac{}{(1 \oplus 2) \otimes (3 \oplus 4) \rightarrow 3 \otimes (3 \oplus 4)} \end{array}$$

A Computation



$$\begin{aligned}(1 \oplus 2) \otimes (3 \oplus 4) &\rightarrow (1 \oplus 2) \otimes (3 \oplus 4) \\ &\rightarrow (1 \oplus 2) \otimes (3 \oplus 4) \\ &\rightarrow 3 \otimes (3 \oplus 4) \\ &\rightarrow 3 \otimes (3 \oplus 4) \\ &\rightarrow 3 \otimes (3 \oplus 4) \\ &\rightarrow 3 \otimes 7 \\ &\rightarrow 21 \\ &\nrightarrow\end{aligned}$$

Therefore, $(1 \oplus 2) \otimes (3 \oplus 4) \rightarrow^* 21$.

Note: For each use of ' $\rightarrow$ ' above, there is a corresponding derivation.

Another Computation



$$\begin{aligned}(1 \oplus 2) \otimes (3 \oplus 4) &\rightarrow (1 \oplus 2) \otimes (3 \oplus 4) \\&\rightarrow (1 \oplus 2) \otimes (3 \oplus 4) \\&\rightarrow (1 \oplus 2) \otimes (3 \oplus 4) \\&\rightarrow (1 \oplus 2) \otimes 7 \\&\rightarrow (1 \oplus 2) \otimes 7 \\&\rightarrow 3 \otimes 7 \\&\rightarrow 21 \\&\nrightarrow\end{aligned}$$

Therefore, $(1 \oplus 2) \otimes (3 \oplus 4) \rightarrow^* 21$.

Confluence?



- ▶ There are many different evaluation sequences starting from the same expression—this is called **non-determinism**.
- ▶ Are they guaranteed to lead to the same outcome?
- ▶ We can optimize the inference rules to impose **determinism**, i.e., some specific evaluation strategy.
- ▶ For example, we can impose a left-to-right evaluation of arguments by changing rules (sumR) and (prodR).

SOS Rules with Evaluation Strategy



$$\frac{(\text{num})}{N \rightarrow n}$$

$$\frac{(\text{sum})}{n_0 \oplus n_1 \rightarrow n} \quad n = n_0 + n_1$$

$$\frac{(\text{sumL}) \quad E_0 \rightarrow E'_0}{E_0 \oplus E_1 \rightarrow E'_0 \oplus E_1}$$

$$\frac{(\text{sumR}) \quad E_1 \rightarrow E'_1}{n_0 \oplus E_1 \rightarrow n_0 \oplus E'_1}$$

Where is the difference?

SOS Rules with Evaluation Strategy



$$\frac{(\text{num})}{N \rightarrow n}$$

$$\frac{(\text{sum})}{n_0 \oplus n_1 \rightarrow n} \quad n = n_0 + n_1$$

$$\frac{(\text{sumL}) \quad E_0 \rightarrow E'_0}{E_0 \oplus E_1 \rightarrow E'_0 \oplus E_1}$$

$$\frac{(\text{sumR}) \quad E_1 \rightarrow E'_1}{n_0 \oplus E_1 \rightarrow n_0 \oplus E'_1}$$

Where is the difference? What about the rest?

(prod)

(prodL)

(prodR)

A Computation



With the adopted evaluation strategy, we have only one possible computation:

$$\begin{aligned}(1 \oplus 2) \otimes (3 \oplus 4) &\rightarrow (1 \oplus 2) \otimes (3 \oplus 4) \\ &\rightarrow (1 \oplus 2) \otimes (3 \oplus 4) \\ &\rightarrow 3 \otimes (3 \oplus 4) \\ &\rightarrow 3 \otimes (3 \oplus 4) \\ &\rightarrow 3 \otimes (3 \oplus 4) \\ &\rightarrow 3 \otimes 7 \\ &\rightarrow 21 \\ &\nrightarrow\end{aligned}$$

Therefore, $(1 \oplus 2) \otimes (3 \oplus 4) \rightarrow^* 21$.



- ▶ A step represents a whole computation:

$$E_0 \longrightarrow n$$

- ▶ How to define the transition relation?
- ▶ Usually simpler than small-step rules.
- ▶ Can correspond to an efficient interpreter!
- ▶ Cannot express non-terminating computations: derivations are possible only for terminating programs.

SOS Rules - Big-step Semantics



$$\frac{(\text{num})}{N \longrightarrow n}$$

$$\frac{(\text{sum}) \quad E_0 \longrightarrow n_0 \quad E_1 \longrightarrow n_1}{E_0 \oplus E_1 \longrightarrow n} \quad n = n_0 + n_1$$

$$(\text{prod})$$

A Derivation



Example

$$\begin{array}{c} \begin{array}{c} \text{(num)} \frac{}{1 \longrightarrow 1} \quad \text{(num)} \frac{}{2 \longrightarrow 2} \\ \text{(sum)} \frac{}{(1 \oplus 2) \longrightarrow 3} \\ \text{(prod)} \frac{}{(1 \oplus 2) \otimes (3 \oplus 4) \longrightarrow 21} \end{array} \quad \begin{array}{c} \text{(num)} \frac{}{3 \longrightarrow 3} \quad \text{(num)} \frac{}{4 \longrightarrow 4} \\ \text{(sum)} \frac{}{(3 \oplus 4) \longrightarrow 7} \end{array} \end{array}$$



- ▶ We define a **domain** and an **interpretation function**.
- ▶ For expressions, the domain is $\mathbb{N}$ and the interpretation is

$$\mathcal{E}[\![\cdot]\!] : Exp \rightarrow \mathbb{N}$$

- ▶ Given E (a piece of syntax), its **denotation** $\mathcal{E}[\![E]\!]$ is a semantic object.



- ▶ We define a **domain** and an **interpretation function**.
- ▶ For expressions, the domain is $\mathbb{N}$ and the interpretation is

$$\mathcal{E}[\![\cdot]\!] : Exp \rightarrow \mathbb{N}$$

- ▶ Given E (a piece of syntax), its **denotation** $\mathcal{E}[\![E]\!]$ is a semantic object.
- ▶ The choice of the domain has immediate consequences:
We already know that expressions are deterministic (every expression has at most one answer) and normalizing (every expression has an answer)
- ▶ Different syntactic categories may require different domains!



$$\mathcal{E}[\![N]\!] = n$$

$$\mathcal{E}[\![E_0 \oplus E_1]\!] = \mathcal{E}[\![E_0]\!] + \mathcal{E}[\![E_1]\!]$$

$$\mathcal{E}[\![E_0 \otimes E_1]\!] = \mathcal{E}[\![E_0]\!] \cdot \mathcal{E}[\![E_1]\!]$$

► **Compositionality Principle:**

The meaning of a composite construct does not depend on the particular form of the constituent constructs, but only on their meanings.



The denotational semantics at work:

$$\begin{aligned}\mathcal{E}[(1 \oplus 2) \otimes (3 \oplus 4)] &= \mathcal{E}[1 \oplus 2] \cdot \mathcal{E}[3 \oplus 4] \\ &= (\mathcal{E}[1] + \mathcal{E}[2]) \cdot (\mathcal{E}[3] + \mathcal{E}[4]) \\ &= (1 + 2) \cdot (3 + 4) \\ &= 21\end{aligned}$$



We have seen different approaches to the semantics of numerical expressions.

Let us consider some angles for comparison:

- ▶ Fundamental properties (termination, determinacy)
- ▶ Induced equivalences
- ▶ Congruences (equivalences under any context)

Comparison



We have seen different approaches to the semantics of numerical expressions:

$$E \rightarrow^* n$$

$$E \longrightarrow n$$

$$\mathcal{E}[[E]] = n$$

Consider a key property: **termination** (aka normalization):

$$\forall E. \exists n. E \rightarrow^* n$$

must be proved

$$\forall E. \exists n. E \longrightarrow n$$

must be proved

$$\forall E. \exists n. \mathcal{E}[[E]] = n$$

immediate

Comparison



Now consider **determinacy**:

$\forall E, n, m. (E \rightarrow^* n \wedge E \rightarrow^* m) \Rightarrow n = m$ must be proved

$\forall E, n, m. (E \longrightarrow n \wedge E \longrightarrow m) \Rightarrow n = m$ must be proved

$\forall E, n, m. (\mathcal{E}[[E]] = n \wedge \mathcal{E}[[E]] = m) \Rightarrow n = m$ immediate

Comparison



Consider consistency:

$$\forall E. \exists n. (E \rightarrow^* n \Leftrightarrow E \longrightarrow n \Leftrightarrow \mathcal{E}[[E]] = n)$$

must be proved

Comparison



Consider **consistency**:

$$\forall E. \exists n. (E \rightarrow^* n \Leftrightarrow E \longrightarrow n \Leftrightarrow \mathcal{E}[[E]] = n)$$

must be proved

Consider **induced equivalences**:

- ▶ Define $E_0 \equiv_s E_1$ iff $\forall n. (E_0 \rightarrow^* n \Leftrightarrow E_1 \rightarrow^* n)$
- ▶ Define $E_0 \equiv_b E_1$ iff $\forall n. (E_0 \longrightarrow n \Leftrightarrow E_1 \longrightarrow n)$
- ▶ Define $E_0 \equiv_d E_1$ iff $\mathcal{E}[[E_0]] = \mathcal{E}[[E_1]]$.

Do these equivalences coincide?



Can we prove or disprove:

- Properties of **specific expressions**, such as, e.g.,

$$2 \otimes 6 \equiv_s 3 \otimes 4$$

- Properties of **generic expressions**, such as, e.g.,

$$\forall E, E_1, E_2. E \otimes (E_1 \oplus E_2) \equiv_d (E \otimes E_1) \oplus (E \otimes E_2)$$

Comparison



- An equivalence is a **congruence** when it is preserved by all constructs of a language. A congruence ensures that equality holds under any **context**.

Comparison



- ▶ An equivalence is a **congruence** when it is preserved by all constructs of a language. A congruence ensures that equality holds under any **context**.
- ▶ A context for numerical expressions is an expression with a 'hole', denoted $\bullet$:

$$C[\bullet] ::= [\bullet] \mid C[\bullet] \oplus E \mid E \oplus C[\bullet] \mid C[\bullet] \otimes E \mid E \otimes C[\bullet]$$

- ▶ This way, given a context $C[\bullet]$ and an expression E , a **filled context** is $C[E]$.

Comparison



- ▶ An equivalence is a **congruence** when it is preserved by all constructs of a language. A congruence ensures that equality holds under any **context**.
- ▶ A context for numerical expressions is an expression with a 'hole', denoted $\bullet$:

$$C[\bullet] ::= [\bullet] \mid C[\bullet] \oplus E \mid E \oplus C[\bullet] \mid C[\bullet] \otimes E \mid E \otimes C[\bullet]$$

- ▶ This way, given a context $C[\bullet]$ and an expression E , a **filled context** is $C[E]$.

Relevant properties:

$\forall E_0, E_1, C[\bullet]. (E_0 \equiv_s E_1 \Rightarrow C[E_0] \equiv_s C[E_1])$ must be proven

$\forall E_0, E_1, C[\bullet]. (E_0 \equiv_b E_1 \Rightarrow C[E_0] \equiv_b C[E_1])$ must be proven

$\forall E_0, E_1, C[\bullet]. (E_0 \equiv_d E_1 \Rightarrow C[E_0] \equiv_d C[E_1])$ immediate

Beyond Numerical Expressions



Suppose that we have expressions with **variables**, denoted $x, y, \dots$:

$$E ::= x \mid N \mid E \oplus E \mid E \otimes E$$

- How to evaluate expressions such as $(x \oplus 4) \otimes y$?

Beyond Numerical Expressions



Suppose that we have expressions with **variables**, denoted $x, y, \dots$:

$$E ::= x \mid N \mid E \oplus E \mid E \otimes E$$

- ▶ How to evaluate expressions such as $(x \oplus 4) \otimes y$?
- ▶ Solution: We need some **memories**

$$\mathbb{M} \triangleq \{\sigma \mid \sigma : X \rightarrow \mathbb{N}\}$$

- ▶ The states of the abstract machines and the interpretation function:

$$\langle E, \sigma \rangle \qquad \mathcal{E}[\![\cdot]\!] : Exp \rightarrow (\mathbb{M} \rightarrow \mathbb{N})$$

How to redefine the various semantics and properties?



The End